GOA UNIVERSITY

Master of Computer Applications (MCA)

KONKANI

SPEECH-TO-TEXT SYSTEM

A Java–Python Application for Converting Spoken Konkani to Devanagari and Roman Script

MINI PROJECT REPORT

Submitted in partial fulfilment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

Academic Year 2026–2027

Submitted by

Sr. No.	Name of Student	Roll Number
1	Rakshita Chari	2636
2	Saisha Prabhu	2642

Under the guidance of

Prof. Hanumant Harichandra Redkar

CERTIFICATE

This is to certify that the mini project report entitled "**Konkani Speech-to-Text System: A Java–Python Application for Converting Spoken Konkani into Devanagari and Roman Script**" is a bonafide record of the work carried out by the students named below, in partial fulfilment of the requirements for the award of the degree of **Master of Computer Applications (MCA)** of **Goa University**, during the academic year 2026–2027.

Name of Student	Roll Number
Rakshita Chari	2636
Saisha Prabhu	2642

The report has not been submitted to any other university or institution for the award of any degree or diploma.

Date: _____ **Place:** _____

Project Guide	Head of Department	External Examiner

DECLARATION

We hereby declare that this mini project report titled "**Konkani Speech-to-Text System**" is the result of our own work. The pretrained speech-recognition model used in the project (AI4Bharat IndicConformer) and the open-source libraries used (FastAPI, PyTorch, Transformers, ONNX Runtime, SoundFile and TorchAudio) are the work of their respective authors and have been acknowledged in the references. Wherever the ideas, text or results of other researchers have been used, they have been cited. This report has not been submitted elsewhere for any degree or diploma.

Rakshita Chari 2636
Saisha Prabhu 2642

Date: _____ **Place:** _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who helped us complete this mini project.

First, we thank our project guide, **Prof. Hanumant Harichandra Redkar**, for the continuous guidance, valuable suggestions and encouragement during every stage of the work, from the first brainstorming sessions to the final testing.

We acknowledge the AI4Bharat initiative at IIT Madras for releasing the IndicConformer speech-recognition models for Indian languages. Without such openly available research, a Konkani speech recognizer would not have been possible within the time of a mini project. We also thank the open-source communities behind Python, Java, FastAPI, PyTorch, Hugging Face Transformers and ONNX Runtime.

We thank our friends and family, and the native Konkani speakers who helped us test the system with their voices. Finally, we thank everyone who taught us patience while we debugged installation errors on two different operating systems.

Rakshita Chari 2636
Saisha Prabhu 2642

ABSTRACT

Konkani is an Indo-Aryan language spoken along the west coast of India, mainly in Goa, coastal Karnataka (the Mangalore region), Maharashtra and Kerala. It is one of the 22 scheduled languages of India, yet it has far fewer digital tools than Hindi or Marathi. It is written in several scripts: Devanagari (the official script in Goa), Roman (known as Romi Konkani), Kannada and Malayalam. Its dialects, such as Goan and Mangalorean Konkani, differ in pronunciation and vocabulary.

This mini project builds a desktop application that lets a person speak in Konkani into a microphone and see the spoken words as text, in both Devanagari and Roman script. The system has two parts. A Java Swing client records the microphone, creates a WAV file and sends it over HTTP to a local server. A Python FastAPI server converts the audio to 16 kHz mono, runs the pretrained AI4Bharat IndicConformer multilingual speech-recognition model with the Konkani language code, and returns the recognized Devanagari text. A rule-based transliteration module, written for this project, then converts the Devanagari text into Roman script. The server returns both texts as JSON, and the Java client displays them.

The report documents the full development process: the initial brainstorming, the ideas that were accepted and rejected, the requirements, the system design with all diagrams, the implementation, the problems met during installation and integration and how they were solved, the testing approach, and the limitations of the system, especially for the Mangalorean dialect. The main learning outcomes are the design of a client–server application, the practical use of a pretrained deep-learning model for inference, and the engineering effort needed to make such a model run on ordinary laptops.

Keywords: Konkani, Automatic Speech Recognition, Conformer, CTC, Transliteration, Devanagari, Romi Konkani, Java Swing, FastAPI, Client–Server

TABLE OF CONTENTS

Certificate ..... **i**
Declaration ..... **ii**
Acknowledgement ..... **iii**
Abstract ..... **iv**
Table of Contents ..... **v**
List of Figures ..... **vii**
List of Tables ..... **viii**
List of Abbreviations ..... **ix**

Chapter 1 Introduction **1**
1.1 Background 1
1.2 About the Konkani Language 1
1.3 Motivation 1
1.4 Problem Statement 2
1.5 Objectives 2
1.6 Scope of the Project 2
1.7 Applications 3
1.8 Organization of the Report 3

Chapter 2 Background and Literature Survey **4**
2.1 Automatic Speech Recognition 4
2.2 Evolution of ASR Approaches 4
2.3 The Conformer Architecture 5
2.4 Speech Recognition for Indian Languages 5
2.5 Evaluation: Word Error Rate 6
2.6 Transliteration 6
2.7 Client–Server Architecture and REST 6
2.8 Summary of the Survey 7

Chapter 3 Brainstorming, Idea Evaluation and Technology Selection **8**
3.1 Project Initiation and Clarifying the Problem 8
3.2 Brainstorming Sessions 8
3.3 Ideas Considered: Accepted, Rejected and Deferred 9
3.4 Comparison of the Speech-Recognition Options 10
3.5 Why the Java–Python Split Was Accepted 11
3.6 Development Phases and Activities 11

Chapter 4 Requirement Analysis and Feasibility Study **13**
4.1 Stakeholders and Users 13
4.2 Functional Requirements 13
4.3 Non-Functional Requirements 14
4.4 Hardware Requirements 14
4.5 Software Requirements 14
4.6 Feasibility Study 15
4.7 Risk Analysis 16
4.8 Assumptions and Constraints 16

Chapter 5 System Design 17
5.1 System Overview 17
5.2 Brainstorming Mind Map 17
5.3 Overall Class Diagram 18
5.4 Sequence Diagram 19
5.5 Class Diagrams of the Two Programs 20

Chapter 6 Implementation 23
6.1 Development Environment 23
6.2 Project Folder Structure 23
6.3 Implementation of the Java Client 23
6.4 Implementation of the Python Server 25
6.5 Implementation of Transliteration 27
6.6 Configuration 27
6.7 How to Run 28

Chapter 7 Problems Faced During Development and Their Solutions 29
7.1 Summary Table 29
7.2 P2 and P3: Missing Modules and the Python Version 30
7.3 P4 and P5: Environment Creation Failures 30
7.4 P6 and P7: Gated Model Access 30
7.5 P8 and P9: ONNX Runtime Path Check 31
7.6 P10: HTTP 422 and the Java Upload 31
7.7 P11: Out-of-Memory Crash of the Java Virtual Machine 31
7.8 P12: Stale Builds and Unsaved Files 32
7.9 Lessons Learned 32

Chapter 8 Testing and Evaluation 33
8.1 Test Strategy 33
8.2 Test Environment 33
8.3 Functional Test Cases 33
8.4 Unit Tests of the Transliteration Module 35
8.5 Accuracy Evaluation Method 35
8.6 Performance Measurement 36
8.7 Test Observations from Development 36

Chapter 9 Working Product and Screenshots 38
9.1 Working Product Screenshot 38

Chapter 10 Limitations, Future Scope and Conclusion 39
10.1 Limitations 39
10.2 Future Scope 39
10.3 Learning Outcomes 40
10.4 Conclusion 40

References 41

Appendix A Java Client Source Code 42

Appendix B Python Server Source Code 43

Appendix C API Specification 44

LIST OF FIGURES

Figure 5.1 System overview of the Konkani Speech-to-Text application.....17
Figure 5.2 Brainstorming mind map of the Konkani Speech-to-Text system.....18
Figure 5.3 Overall class diagram.....19
Figure 5.4 Sequence diagram of one recording and transcription.....20
Figure 5.5 Class diagram of the Java client.....21
Figure 5.6 Class and module diagram of the Python backend.....22
Figure 9.1 The Konkani Speech-to-Text client showing two test recordings.....38

LIST OF TABLES

Table 3.1 Brainstorming sessions and their outcomes....8
Table 3.2 Ideas considered, with decisions and reasons....9
Table 3.3 Weighted comparison of speech-recognition options....10
Table 3.4 Development phases, activities and outputs....11
Table 4.1 Functional requirements....13
Table 4.2 Non-functional requirements....14
Table 4.3 Hardware requirements....14
Table 4.4 Software requirements....14
Table 4.5 Risk analysis and mitigation....16
Table 6.1 Development environment....23
Table 6.2 Configuration settings....27
Table 7.1 Summary of problems, causes and solutions....29
Table 8.1 Levels of testing....33
Table 8.2 Test environment....33
Table 8.3 Functional test cases....34
Table 8.4 Unit tests of the transliteration module....35
Table 8.5 Word and character error rates by dialect (to be completed)....36
Table 8.6 Performance measurements (to be recorded)....36
Table 10.1 Future enhancements and their benefits....39
Table 10.2 API endpoints....44
Table 10.3 Status codes returned by POST /transcribe....44

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ASR	Automatic Speech Recognition
API	Application Programming Interface
CTC	Connectionist Temporal Classification
RNN-T	Recurrent Neural Network Transducer
STT	Speech-to-Text
TTS	Text-to-Speech
WER	Word Error Rate
CER	Character Error Rate
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
WAV	Waveform Audio File Format
PCM	Pulse-Code Modulation
GUI	Graphical User Interface
JDK	Java Development Kit
ONNX	Open Neural Network Exchange
HF	Hugging Face
MFCC	Mel-Frequency Cepstral Coefficients
DFD	Data Flow Diagram
UML	Unified Modeling Language

CHAPTER 1

Introduction

1.1 Background

Speech is the most natural way for people to communicate, and speech-to-text technology (also called Automatic Speech Recognition, or ASR) turns spoken words into written text. Today, ASR is part of phone keyboards, voice assistants, meeting transcription and accessibility tools. For languages with many speakers and large datasets, such as English, the quality is very high. For many Indian languages, especially those with fewer speakers and less recorded data, the tools are still limited.

Konkani is one such language. It is the official language of the state of Goa and is also spoken in Karnataka, Maharashtra, Kerala and by communities abroad. It was included in the Eighth Schedule of the Constitution of India in 1992. Although it has a rich literature and a living spoken tradition, the number of ready-to-use speech technologies for it is small.

1.2 About the Konkani Language

Konkani belongs to the Indo-Aryan family. A few facts that matter for this project:

1. **Multiple scripts.** Konkani is written in Devanagari (the official script in Goa), in the Roman script (Romi Konkani, widely used by the Catholic community and in church and popular writing), in the Kannada script (commonly used around Mangalore), and in Malayalam and Perso-Arabic scripts in some communities. There is no single universal spelling in Roman script.
2. **Many dialects.** Goan Konkani (with sub-dialects such as Bardezi and Salcete), Mangalorean Konkani, Karwari, Malvani and Cochin Konkani differ in vocabulary, pronunciation and sometimes in grammar. A speaker from Mangalore and a speaker from Goa will often pronounce the same word differently.
3. **Sounds that need care.** Konkani has nasal vowels and sounds, aspirated and unaspirated consonants, and the retroflex consonants common in Indian languages. These make recognition and transliteration harder than for languages with simpler sound systems.

Because of these facts, a Konkani speech-to-text tool has two separate jobs: recognizing the speech and then showing the text in the script the user prefers.

1.3 Motivation

The project was motivated by the following observations:

- A person who speaks Konkani cannot easily dictate a message or a document in Konkani on a computer, because common dictation tools do not support the language well.
- Many Konkani speakers type Konkani in Roman script on their phones, but others want Devanagari. A system that offers both is more useful.
- The Mangalorean dialect is under-represented in public datasets compared with Goan Konkani, so it makes a good case study of the accent and dialect problem that affects all low-resource languages.
- Recent open research from AI4Bharat has produced pretrained models for 22 Indian languages, which makes it realistic for students to build a working application on top of them.

1.4 Problem Statement

> **Problem.** To design and develop a desktop application in which a user speaks in Konkani, in any dialect including Goan and Mangalorean, into a microphone, and the application displays the spoken content as text in Devanagari script and Roman script (Romi Konkani).

1.5 Objectives

The objectives of the project are:

1. To study how modern speech-recognition systems work and which models support Konkani.
2. To record live speech from a microphone using Java and send it to a processing server.
3. To build a Python web service that converts the received audio into Konkani text in Devanagari script using a pretrained model.
4. To convert the Devanagari text into Roman script with a rule-based transliteration module.
5. To display both outputs to the user in a simple graphical interface.
6. To document the design, the problems faced, the tests carried out and the limitations, especially for the Mangalorean dialect.

1.6 Scope of the Project

In scope

- Live recording from the default microphone and transcription after the recording stops.
- Konkani recognition using the pretrained IndicConformer model.
- Output in Devanagari and in machine-transliterated Roman script.

- Running on a single computer (Windows or macOS) with the server on `127.0.0.1`.

Out of scope

- Training or fine-tuning a speech model (identified as future work).
- Real-time word-by-word streaming recognition.
- Kannada-script output.
- Cloud deployment and multi-user support.
- A mobile application (considered and deferred, see Chapter 3).

1.7 Applications

- Dictation of Konkani messages, notes and documents.
- Helping journalists and writers prepare Konkani text from interviews.
- Subtitling and archiving of Konkani audio, such as folk songs, sermons and oral histories.
- Accessibility support for people who find typing in Konkani difficult.
- A teaching and research base for building a larger Konkani speech dataset.

1.8 Organization of the Report

Chapter 2 surveys the background technology. Chapter 3 records our brainstorming and the accepted and rejected ideas. Chapter 4 lists requirements and checks feasibility. Chapter 5 presents the design with diagrams. Chapter 6 describes the implementation. Chapter 7 describes the problems faced during development. Chapter 8 covers testing and evaluation. Chapter 9 presents the working product. Chapter 10 gives the limitations, future scope and conclusion. The appendices contain the source code and the API specification.

CHAPTER 2

Background and Literature Survey

2.1 Automatic Speech Recognition

Automatic Speech Recognition is the task of mapping an audio waveform to the sequence of words that was spoken. The input is a signal sampled many thousand times per second, and the output is text. The difficulty comes from variation: different speakers, accents, speaking speeds, background noise, microphones and recording conditions all change the signal, but the system should still produce the same words.

2.1.1 Digital audio basics

A microphone produces a continuous voltage that the computer samples at a fixed rate. The three key properties are:

- **Sampling rate:** the number of samples per second. Our Java client records at 44,100 Hz, the standard CD rate. Most speech models, including the one we use, expect 16,000 Hz, because the useful information in human speech lies mostly below 8 kHz (by the Nyquist rule, a 16 kHz rate can represent frequencies up to 8 kHz).
- **Bit depth:** the precision of each sample. We use 16 bits.
- **Channels:** mono (one) or stereo (two). Speech recognition uses mono.

The raw samples are stored without compression in PCM form and wrapped in a WAV file, which adds a small header describing the format.

2.1.2 Features: spectrograms and MFCCs

A neural network does not read raw samples directly. The audio is cut into short overlapping frames, typically about 25 milliseconds long with a step of 10 milliseconds, and each frame is converted into a set of numbers that describe how much energy it has at each frequency. Stacking these frames over time gives a spectrogram, which looks like a picture of the sound. Using a scale that mimics human hearing (the mel scale) gives the log-mel spectrogram, and a further transformation gives MFCCs. The model then treats the spectrogram like an image-like sequence and learns to map it to characters or word pieces.

2.2 Evolution of ASR Approaches

1. **Template matching (1950s–1970s).** Early systems compared the input with stored templates of single words, using dynamic time warping. They worked only for small vocabularies and single speakers.
2. **HMM–GMM systems (1980s–2000s).** Hidden Markov Models modelled how speech sounds change over time, and Gaussian Mixture Models described the sound

of each state. Toolkits such as CMU Sphinx and Kaldi used this approach together with a pronunciation dictionary and a language model.

3. **DNN–HMM hybrids (2010s).** Deep neural networks replaced the GMMs and improved accuracy greatly, but the system still needed separate acoustic, pronunciation and language components.
4. **End-to-end models (late 2010s onward).** A single network maps audio directly to text. The main training and decoding methods are:
   - **CTC (Connectionist Temporal Classification),** introduced by Graves et al. in 2006, which lets the network output a character for each frame, including a special "blank", and then collapses repeats and blanks. It does not need frame-level alignments in the training data.
   - **Attention-based encoder–decoder models,** which learn to attend to the relevant part of the audio while writing each output token.
   - **RNN-Transducer (RNN-T),** introduced by Graves in 2012, which combines an audio encoder with a prediction network that models previous outputs and is well suited to streaming.
5. **Self-supervised and large-scale models.** Models such as wav2vec 2.0 (Baevski et al., 2020) learn from large amounts of unlabelled audio and then need only a small labelled set. Whisper (Radford et al., 2022) was trained on a very large collection of weakly supervised multilingual data.

2.3 The Conformer Architecture

The Conformer (Gulati et al., Interspeech 2020) is the encoder design used by the IndicConformer models. A Conformer block combines two ideas:

- **Self-attention,** which captures global context across the whole utterance (for example, how a word at the end relates to a word at the start).
- **Convolution,** which captures local patterns in the spectrogram (for example, the short burst of a consonant).

Each block contains a feed-forward layer, a multi-head self-attention layer, a convolution module and a second feed-forward layer, with residual connections and layer normalization. Stacking many such blocks gives a strong acoustic encoder. On top of the encoder, either a CTC layer or an RNN-T decoder produces the text. The version we use has about 600 million parameters and supports both decoding modes.

2.4 Speech Recognition for Indian Languages

Indian languages pose particular challenges: many languages and dialects, code-mixing with English, limited labelled data for smaller languages, and several scripts. The AI4Bharat group at IIT Madras has released open datasets and models for this area. Javed et al. (2022)

described IndicWav2Vec, a family of models pretrained on thousands of hours of Indian-language speech. Later work introduced the large IndicVoices dataset, collected from many districts and including natural, spontaneous speech, and the IndicConformer models trained on it. The multilingual IndicConformer model (600 million parameters) supports the 22 scheduled languages of India, which includes Konkani, and is available through Hugging Face. Its Konkani language code is `kok`.

The quality of such a model is not equal across dialects. The recordings available for Goan Konkani are more numerous than those for Mangalorean Konkani, so we expect higher error rates for the latter. This is discussed in the limitations (Chapter 10).

2.5 Evaluation: Word Error Rate

The standard measure for ASR is the Word Error Rate:

$$WER = (S + D + I) / N$$

where S is the number of substituted words, D the number of deleted words, I the number of inserted words, and N the number of words in the reference (correct) transcript. A WER of 0.20 means that, roughly, one word in five is wrong. The Character Error Rate (CER) is defined in the same way at the character level and is often more informative for languages with long, inflected words.

2.6 Transliteration

Transliteration converts text from one script to another while keeping the pronunciation, as opposed to translation, which changes the language. Two broad approaches exist:

- **Rule-based:** a table maps each source letter or sign to a target string. It is fast, transparent and needs no training data, but cannot learn spelling conventions.
- **Statistical or neural:** a model such as AI4Bharat's IndicXlit learns the mapping from examples of word pairs. It can capture common spellings but needs a heavy software stack.

Devanagari is an abugida: each consonant letter carries an inherent vowel "a" unless it is changed by a vowel sign (matra) or removed by the virama sign. This is the main complication in the rule-based approach and is covered in Section 6.5.

The lack of a single Roman spelling standard for Konkani means that any Roman output is one of several acceptable spellings. For this reason the application labels its Roman output as a machine transliteration.

2.7 Client–Server Architecture and REST

In a client–server application, a client program sends requests and a server program processes them and sends back responses. They communicate through a protocol, and the

most common one for web services is HTTP. A REST-style API exposes operations as URLs, such as `POST /transcribe`. Data is exchanged in a simple text format, most commonly JSON. To upload a file, a client uses an HTTP body of type `multipart/form-data`, which splits the body into parts separated by a boundary string; each part has a name and can carry binary content such as a WAV file.

This architecture suits our project because the user interface (best written in Java) and the machine-learning code (best written in Python) can be developed and tested separately and joined only through a small, well-defined API.

2.8 Summary of the Survey

The survey shows that (a) modern end-to-end Conformer models give strong recognition for Indian languages, (b) an open pretrained model for Konkani exists, (c) building a recognizer from scratch is beyond the resources of a mini project, and (d) transliteration into Roman script can be handled by simple rules. These conclusions shaped the technology choices in the next chapter.

CHAPTER 3

Brainstorming, Idea Evaluation and Technology Selection

This chapter records how the project took shape: the first discussions, the ideas that were proposed, the ideas that were accepted or rejected and the reasons, and the way the final technology stack was chosen. We kept this record because the project did not follow a straight line. Several early ideas had to be dropped when we tested them against real constraints such as language support, installation effort and laptop memory.

3.1 Project Initiation and Clarifying the Problem

The starting point was the requirement for a Java-based mini project in which a user speaks Konkani, in any accent or dialect, and the system produces text in Devanagari or Romi Konkani. In the first discussions the requirement was described as “converting text to speech”. On analysis, we found this wording was reversed. Text-to-speech (TTS) converts written text into spoken audio, whereas the project needs speech-to-text (STT), which converts spoken audio into written text. All later work uses the correct term, and the difference is highlighted here because the two technologies use different models and different data.

The second clarification concerned the output. Konkani can be written in several scripts, so the team agreed that the required output is Devanagari (the official script in Goa), with Roman (Romi) as an equally acceptable second form. Kannada script was left out of scope.

The third clarification concerned the dialects. The requirement says “any dialect, such as Mangalorean Konkani”. We recognized early that a student project cannot guarantee equal accuracy for every dialect, so the objective was refined to: support Konkani speech in general, and measure and report honestly how well it works for the Goan and Mangalorean dialects.

3.2 Brainstorming Sessions

The brainstorming was done in a series of working sessions. Each session had a question to answer.

Table 3.1: *Brainstorming sessions and their outcomes*

Session	Question discussed	Main outcome
1	What exactly is the problem and who is the user?	Corrected STT vs TTS; agreed outputs: Devanagari and Roman

Session	Question discussed	Main outcome
2	Which approaches exist for speech recognition of Konkani?	List of fifteen candidate ideas (Section 3.3)
3	Which ideas are feasible in a mini-project time frame?	Shortlist: pretrained model + Java client + Python server
4	How will the Java client and the model communicate?	Client–server design over HTTP with JSON replies
5	How will Roman output be produced?	Rule-based transliteration after Devanagari recognition
6	What can go wrong and how do we test it?	Risk list and test plan (Chapters 4 and 8)
7	What do we do when the installation fails on a laptop?	Fallback plans and the problem log (Chapter 7)

3.3 Ideas Considered: Accepted, Rejected and Deferred

The following table lists every idea that was discussed, the decision and the reason for it.

Table 3.2: *Ideas considered, with decisions and reasons*

ID	Idea	Decision	Reason
I1	Build a text-to-speech system for Konkani	**Rejected**	Based on a wording error. The real need is speech-to-text.
I2	Use an offline Java speech library such as CMU Sphinx or Vosk	**Rejected**	To the best of our knowledge there is no ready acoustic model for Konkani; building one needs hundreds of hours of transcribed speech.
I3	Use OpenAI Whisper	**Rejected**	Konkani is not among the languages Whisper officially lists. Forcing a related language such as Hindi or Marathi would return the wrong words.
I4	Use a commercial cloud speech API	**Rejected**	Needs internet and an account, may involve cost, sends private audio to a third party, and Konkani support is uncertain.
I5	Train a new model from scratch	**Rejected**	Needs very large labelled data and GPU time, far beyond a mini project.
I6	Fine-tune an existing model on our own Mangalorean recordings	**Deferred**	Valuable for dialect accuracy, but needs recorded and transcribed data and GPU time. Listed as future work.
I7	Use the Konkani-specific AI4Bharat model through the NVIDIA NeMo toolkit	**Rejected** for this build	NeMo is difficult to install on Windows, needs particular Python and PyTorch versions, and the early backend draft also needed FFmpeg.

ID	Idea	Decision	Reason
I8	Use the multilingual AI4Bharat IndicConformer (600M) from Hugging Face with language code `kok`	**Accepted**	Supports Konkani, installs with ordinary pip packages, runs on CPU, and returns Devanagari text.
I9	Use the neural transliteration engine IndicXlit for Roman output	**Rejected**	Depends on fairseq, which often fails to build on Windows and on new Python versions; heavy for a small feature.
I10	Write our own rule-based Devanagari-to-Roman transliterator	**Accepted**	Small, transparent, no extra dependencies, easy to explain and modify.
I11	Build the front end as an Android application	**Deferred**	Needs emulator networking and permission handling and was not required for the Java lab. The same server API can serve an Android client later.
I12	Build the front end as a web page	**Rejected**	The course project is a Java project.
I13	Run the model directly inside Java with ONNX Runtime	**Rejected**	We would have to rewrite audio feature extraction, tokenization and decoding in Java.
I14	Real-time streaming recognition (words appear while speaking)	**Deferred**	Needs a streaming model setup and a different client design.
I15	Output in Kannada script	**Out of scope**	Not required; the model returns Devanagari.

3.4 Comparison of the Speech-Recognition Options

To choose the recognition engine objectively, five options were scored against five criteria. The weights express how important each criterion was to the project. The scores are the team's judgement at the time of selection (1 = poor, 5 = excellent).

Table 3.3: *Weighted comparison of speech-recognition options*

Criterion	Weight	Whisper	Cloud API	IndicConformer (HF)	NeMo Konkani	Train from scratch
Konkani support	30	1	2	5	5	3
Ease of installation on student laptops	25	4	4	4	2	1
Cost	15	5	2	5	5	3
Works offline (privacy)	15	5	1	5	5	5
Accuracy potential	15	2	3	4	4	2
Weighted score (out of 5)	**100**	**3.10**	**2.50**	**4.60**	**4.10**	**2.65**

The weighted score is the sum of (weight × score) divided by 100. For example, for IndicConformer: (30×5 + 25×4 + 15×5 + 15×5 + 15×4) / 100 = 4.60. The multilingual IndicConformer model obtained the highest score and was selected.

3.5 Why the Java–Python Split Was Accepted

Two alternatives were weighed for the overall structure.

1. **Everything in Java.** This would need the neural model to run in Java, which means reimplementing preprocessing and decoding. It was rejected (idea I13).
2. **Everything in Python.** Python has good GUI libraries, but the course requirement and the team's skills point to Java for the application layer.

The accepted solution joins the two: Java for the user interface and microphone, Python for the machine-learning model, and a small HTTP API between them. This also gives a useful side benefit: the same Python server can later be used by an Android app or a web page without changing the model code.

3.6 Development Phases and Activities

The table below lists all the activities carried out during the project.

Table 3.4: *Development phases, activities and outputs*

Phase	Activities	Output
1. Problem study	Clarified STT vs TTS; defined outputs and scope	Problem statement, objectives
2. Literature and tool survey	Studied ASR, Conformer, AI4Bharat models, transliteration	Chapter 2
3. Idea evaluation	Listed and scored alternatives	Tables 3.2 and 3.3
4. Requirement analysis	Wrote functional and non-functional requirements, feasibility, risks	Chapter 4
5. Design	Architecture, data-flow, use-case, sequence and flow diagrams, API design	Chapter 5
6. Java client development	GUI, microphone capture, WAV creation, HTTP upload, JSON parsing	`KonkaniLiveSpeechApp.java`
7. Python server development	FastAPI app, model loading, audio preprocessing, transliteration	`main.py`
8. Environment set-up and integration	Installed packages, solved version and access problems on Windows and macOS	Chapter 7
9. Testing	Functional tests, error tests, accuracy evaluation plan	Chapter 8
10. Documentation	Report, user manual, API specification	This report

CHAPTER 4

Requirement Analysis and Feasibility Study

4.1 Stakeholders and Users

- **End user:** a Konkani speaker who wants to see his or her speech as text. The user needs no technical knowledge beyond clicking two buttons.
- **Developer / maintainer:** the project team, who may extend the system with new models or a new front end.
- **Evaluator:** the guide or examiner, who needs a clear, working demonstration and a documented design.

4.2 Functional Requirements

Table 4.1: *Functional requirements*

ID	Requirement
FR1	The system shall record audio from the default microphone when the user clicks **Start recording**.
FR2	The system shall stop recording and begin transcription when the user clicks **Stop & transcribe**.
FR3	The system shall reject recordings that are too short and tell the user.
FR4	The client shall save the recording as a WAV file and send it to the server over HTTP.
FR5	The server shall convert the audio to 16 kHz mono before recognition.
FR6	The server shall recognize Konkani speech and return the text in Devanagari script.
FR7	The server shall convert the Devanagari text to Roman script by transliteration rules.
FR8	The server shall return both texts in one JSON response with the keys `devanagari` and `roman`.
FR9	The client shall display both texts in separate read-only text areas.
FR10	The client shall show clear messages when the microphone cannot be opened, when the server is not running, and when transcription fails.
FR11	The server shall provide a health-check endpoint that reports whether the model is loaded.
FR12	The client shall delete the temporary WAV file after use.

4.3 Non-Functional Requirements

Table 4.2: *Non-functional requirements*

Category	Requirement
Performance	After the model is loaded, a short recording should be transcribed in a time suitable for interactive use on a laptop CPU. The first request may be slower because of warm-up.
Usability	Two main buttons, a status line and two output boxes; messages in plain language.
Reliability	Errors on either side must not crash the program; the user receives a message.
Privacy	Audio is processed on the user's own computer (`127.0.0.1`). No audio is sent to any outside service after the model is downloaded.
Portability	Works on Windows and macOS with Python and Java installed.
Maintainability	The client and the server are separate programs joined by a documented API, so either can be replaced.
Extensibility	The language code and the decoder (CTC or RNN-T) are configurable on the server.
Cost	Only free and open-source software.

4.4 Hardware Requirements

Table 4.3: *Hardware requirements*

Item	Minimum	Recommended
Processor	64-bit dual core	Quad core or better (the test machine had an 8-core / 16-thread AMD Ryzen 7)
Memory (RAM)	8 GB	16 GB
Free disk space	6 GB	10 GB (model files about 2.5 GB, plus Python libraries)
Microphone	Built-in or wired headset	Wired headset with microphone
Graphics card	Not required	Optional; the model runs on the CPU
Network	Needed once for the model download	Not needed afterwards

4.5 Software Requirements

Table 4.4: *Software requirements*

Software	Version / details	Purpose
Operating system	Windows 10/11 or macOS	Host system

Software	Version / details	Purpose
Java Development Kit	JDK 11 or newer (JDK 17 tested)	Compile and run the client; `java.net.http` needs Java 11+
Python	3.10 to 3.12 (3.13 or newer is not recommended for the ML libraries)	Server
FastAPI and Uvicorn	Latest	Web server and API
python-multipart	Latest	Reading file uploads
PyTorch and TorchAudio	Latest compatible	Tensors and resampling
Transformers	Latest	Loading the Hugging Face model
ONNX Runtime	Latest compatible	Running the exported model parts
SoundFile and NumPy	Latest	Reading WAV audio
Hugging Face account	Free, with access granted to the gated model	Model download
Code editor	Visual Studio Code	Development

4.6 Feasibility Study

Technical feasibility. A pretrained model that supports Konkani is openly available, the libraries are mature and run on ordinary laptops without a GPU, and Java has built-in audio and HTTP classes. The main risks are installation problems and the gated model access, both of which were solved (Chapter 7). The project is technically feasible.

Economic feasibility. All software is free and open source. The only resources used were the team's own laptops, a microphone and an internet connection for a one-time download. The cost is effectively zero.

Operational feasibility. The user starts the server once and then uses a simple window with two buttons. No training or special skill is needed. The system fits the intended use of dictation and demonstration.

Schedule feasibility. Using a pretrained model removes the longest task (training), so the project fits the mini-project period. The schedule risk lies in debugging environments, which is why Chapter 7 documents the problems met.

Legal and ethical feasibility. The model is released by AI4Bharat for use under the terms shown on its Hugging Face page, which the user must accept before downloading. Speech recordings made for testing should be made with the speaker's consent and kept private.

4.7 Risk Analysis

Table 4.5: *Risk analysis and mitigation*

Risk	Likelihood	Impact	Mitigation
Model download is blocked until access is approved	Medium	High	Request access early; keep a copy of the downloaded model folder
Library versions are incompatible with the newest Python	High	High	Use Python 3.12 and a clean virtual environment
Low accuracy for the Mangalorean dialect	High	Medium	State the limitation; measure WER on test recordings; plan fine-tuning
Laptop runs out of memory while loading the model	Medium	High	Close other programs before starting; restart the computer; increase the paging file
Microphone permission or device problems	Medium	Medium	Clear error dialog; instructions in the user manual
No standard Roman spelling for Konkani	Certain	Low	Label the output as a machine transliteration
Server not running when the client is started	High	Low	Client shows a message telling the user to start the server first

4.8 Assumptions and Constraints

- The user speaks clearly, in a quiet room, for a few seconds at a time.
- The server and the client run on the same computer.
- The model supports only the languages it was trained on; the Konkani code `kok` is used.
- Recognition happens after the recording ends, not while the user is still speaking.

CHAPTER 5

System Design

This chapter presents the design of the system with the help of diagrams: the overall architecture, the brainstorming mind map, the overall class diagram, the sequence of operations for one recording, and the class and module diagrams of the Java client and the Python backend.

5.1 System Overview

The system has two programs joined by a small HTTP API. The user speaks to the Java Swing client, which records the audio and sends it with a `POST /transcribe` request to the FastAPI backend on `127.0.0.1:8000`. The backend runs the IndicConformer 600M model (CTC or RNN-T decoding) and returns a JSON reply with the keys `devanagari` and `roman`, which the client displays.



Figure 5.1: *System overview of the Konkani Speech-to-Text application*

5.2 Brainstorming Mind Map

The mind map summarizes the outcome of the brainstorming sessions: the problem, the intended users, the main parts of the front end and the back end, the expected challenges and the ideas kept for the future.

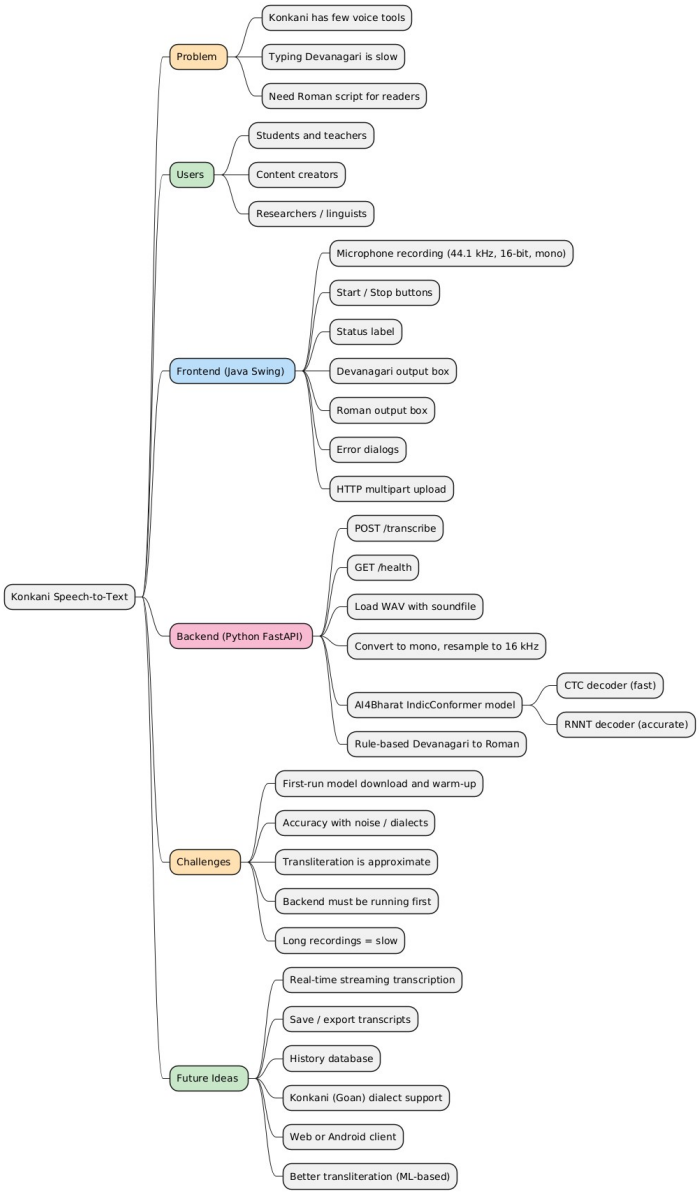


Figure 5.2: *Brainstorming mind map of the Konkani Speech-to-Text system*

5.3 Overall Class Diagram

The overall class diagram shows the main entities of the system and how they relate: a user makes recordings, a recording produces a transcription, a transcription is made with a model configuration, and transliteration rules are applied to the recognized text to give the Roman output.

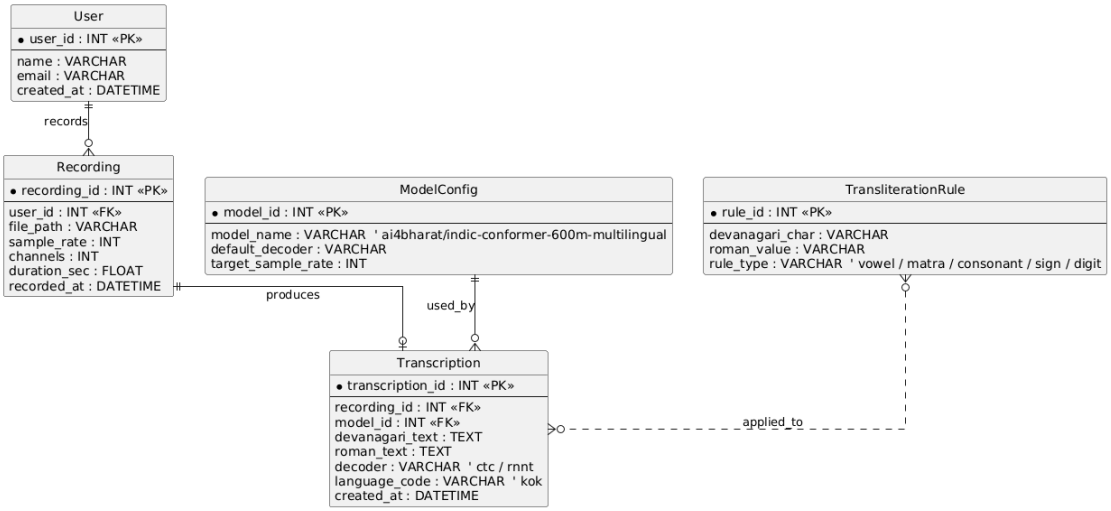


Figure 5.3: *Overall class diagram*

5.4 Sequence Diagram

The sequence diagram follows one complete use of the application. At start-up the server loads the model once. When the user clicks **Start recording**, the client opens the microphone (44.1 kHz, 16-bit, mono) and reads audio in a loop. On **Stop & transcribe** the client stops the recording, writes a WAV file and rejects it if it is shorter than 2,048 bytes. Otherwise a `SwingWorker` posts the file to `/transcribe`. The server answers with HTTP 503 if the model is not loaded and HTTP 400 if the audio is too short; if all is well it converts the audio to 16 kHz mono, runs the model with language code `kok`, transliterates the result to Roman and returns HTTP 200 with both texts. The client then displays them and deletes the temporary WAV file. If the server cannot be reached, a “Cannot reach backend” message is shown.

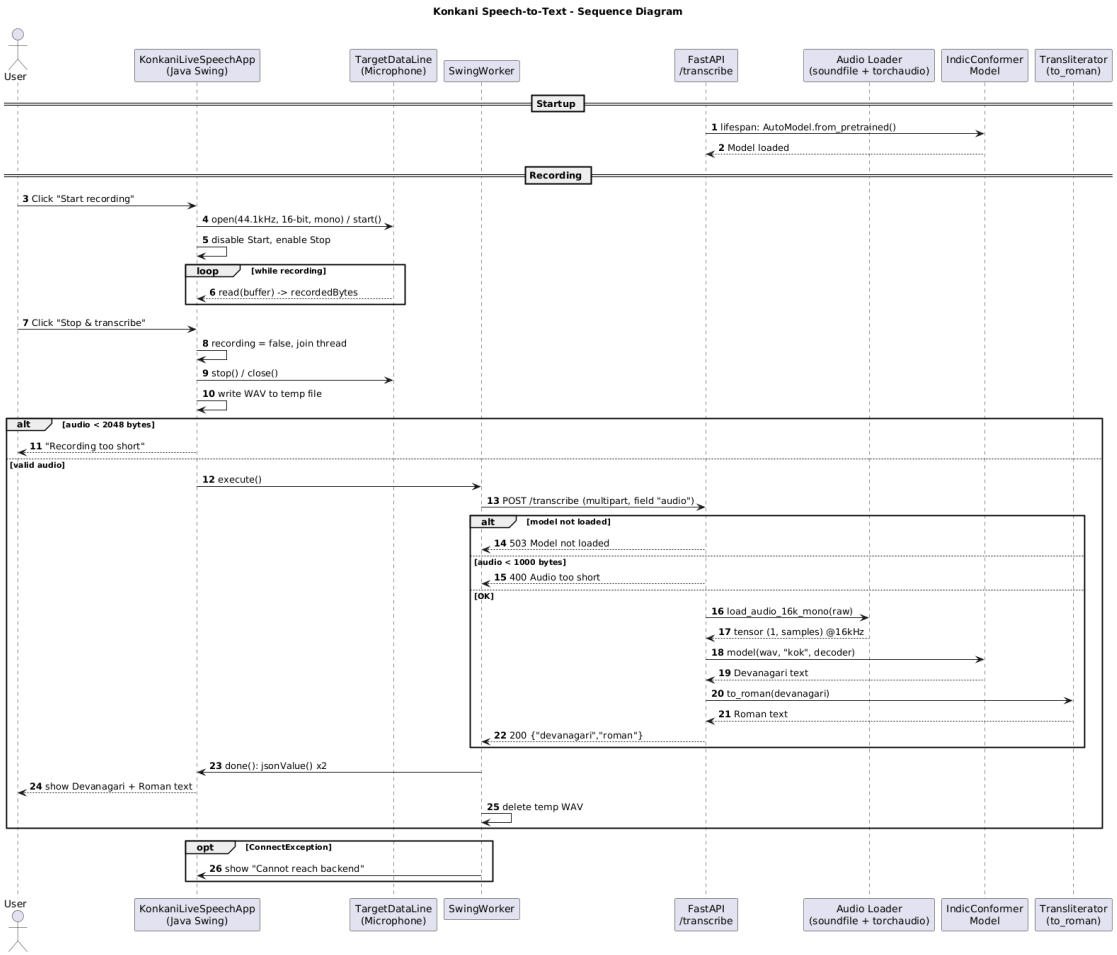


Figure 5.4: *Sequence diagram of one recording and transcription*

5.5 Class Diagrams of the Two Programs

5.5.1 Java client

The Java client is a single class, `KonkaniLiveSpeechApp`, which extends `JFrame`. It records from a `TargetDataLine` using an `AudioFormat`, and creates an anonymous `SwingWorker` that uses `HttpClient` and `HttpRequest` to talk to the server.

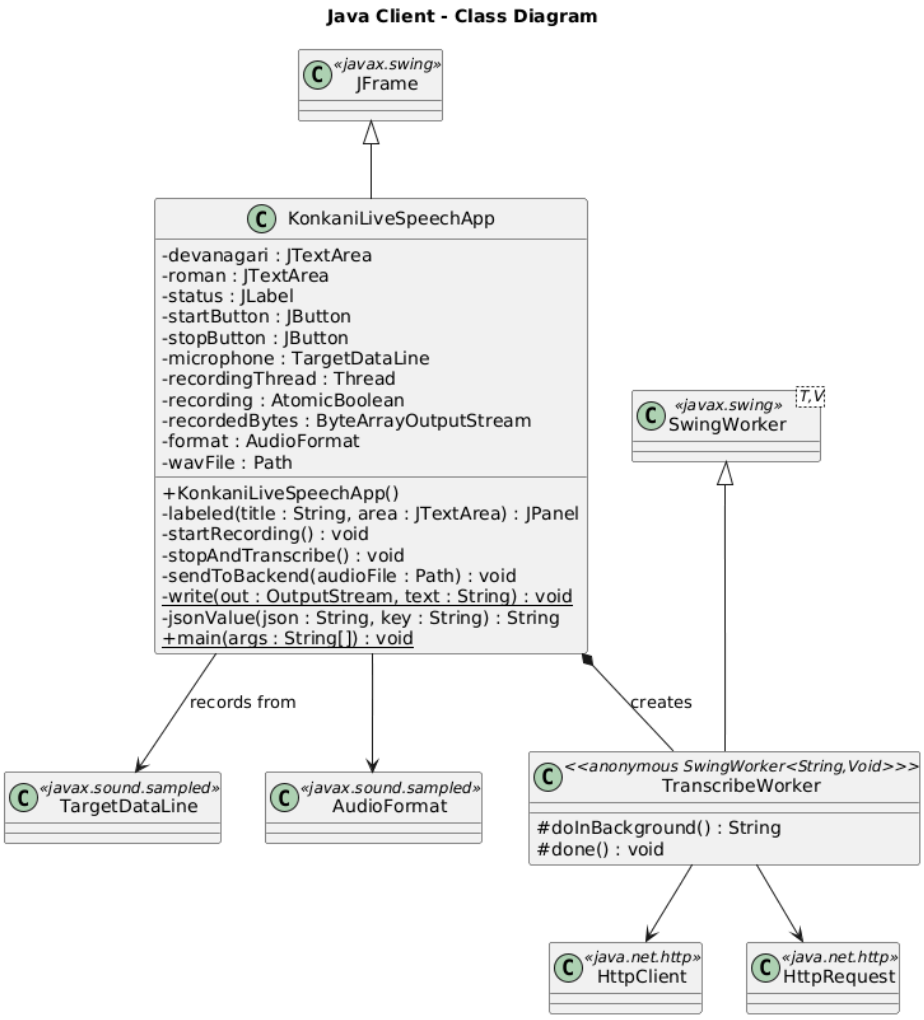


Figure 5.5: *Class diagram of the Java client*

5.5.2 Python backend

The Python backend is organized as modules. `main.py` holds the FastAPI application with the `lifespan`, `health` and `transcribe` functions. It uses the audio-loading module (SoundFile and TorchAudio), the transliterator module with its rule tables, and the Hugging Face `AutoModel` that wraps IndicConformer.

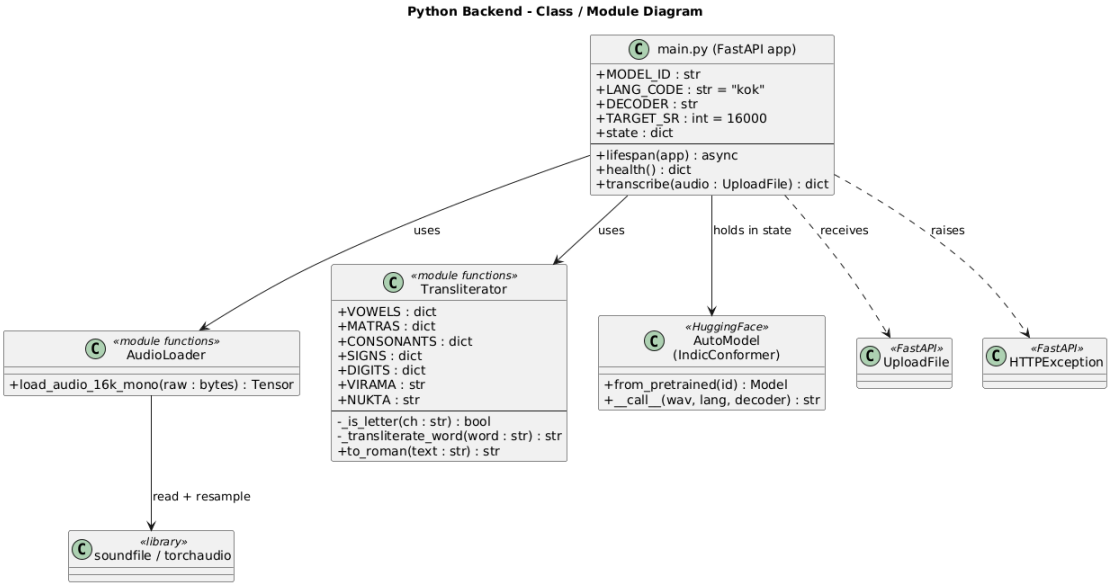


Figure 5.6: *Class and module diagram of the Python backend*

CHAPTER 6

Implementation

This chapter explains how each part of the design was turned into working code. The complete source is in Appendices A and B; here we explain the important parts and the reasons behind them.

6.1 Development Environment

Table 6.1: *Development environment*

Item	Details
Computers	A Windows 11 laptop (AMD Ryzen 7, 16 threads, about 14 GB RAM) and an Apple MacBook Air; the system was run on both
Editor	Visual Studio Code with the Java and Python extensions
Java	JDK 17 or later; `javac` and `java` run from a terminal
Python	3.12 (3.13 was found unsuitable, see Chapter 7)
Server stack	FastAPI, Uvicorn, python-multipart
ML stack	PyTorch, TorchAudio, Transformers, ONNX Runtime
Audio libraries	SoundFile, NumPy
Model	`ai4bharat/indic-conformer-600m-multilingual` from Hugging Face

6.2 Project Folder Structure

```
konkani_speech_to_text/
├── README.txt
├── java/
│   ├── KonkaniLiveSpeechApp.java    (the Swing client)
│   └── run_java.bat                 (compile and run on Windows)
└── py/
    ├── main.py                      (the FastAPI server)
    ├── requirements.txt             (Python packages)
    ├── install.bat, run.bat         (Windows helpers)
    └── model_files/                 (created on first run; about 2.5 GB)
```

6.3 Implementation of the Java Client

6.3.1 Building the window

The constructor creates the `JFrame`, sets a size of 760 × 600 pixels and centres it. Two `JTextArea` objects are created with line wrap and word wrap turned on and are made non-editable, because they only show results. A `FlowLayout` panel at the top holds the two

buttons and the status label. A `GridLayout(2, 1)` panel in the centre stacks the two result boxes, each wrapped by the helper `labeled()` into a titled, scrollable panel. A footer label reminds the user of the server address. The stop button starts disabled, so the user cannot press it before recording begins.

6.3.2 Capturing audio

```
format = new AudioFormat(44100.0f, 16, 1, true, false);
DataLine.Info info = new DataLine.Info(TargetDataLine.class, format);
microphone = (TargetDataLine) AudioSystem.getLine(info);
microphone.open(format);
microphone.start();
```

The format 44,100 Hz, 16-bit, mono, signed, little-endian is supported by almost every microphone driver, which is why it was chosen instead of recording at 16 kHz directly. Resampling to 16 kHz is done on the server. After `start()`, a new thread repeatedly executes `microphone.read(buffer, 0, buffer.length)` into a 4096-byte buffer and appends the bytes to `recordedBytes`. The `while (recording.get())` condition lets the main thread end the loop by setting the flag to false. The write is placed inside a `synchronized (recordedBytes)` block so that the recorder thread and the main thread never touch the stream together.

6.3.3 Stopping safely

The order of operations in `stopAndTranscribe()` matters:

1. Set `recording` to false so the loop will end.
2. Wait for the recorder thread with `join(1500)`.
3. Only then call `microphone.stop()` and `microphone.close()`.

In the first version the line was closed before the thread had finished, which can make `read()` fail. The corrected order is one of the improvements made during review.

6.3.4 Creating the WAV file

The captured bytes are wrapped in an `AudioInputStream`, whose length is the number of bytes divided by the frame size, and `AudioSystem.write(..., AudioFileFormat.Type.WAVE, file)` writes a standard WAV file into the system temporary folder. If the recording is shorter than 2,048 bytes, the method stops and tells the user.

6.3.5 Uploading with multipart/form-data

Java's `HttpClient` has no built-in helper for multipart bodies, so the body is assembled by hand:

```
String boundary = "----JavaBoundary" + System.currentTimeMillis();
write(body, "--" + boundary + "\r\n");
write(body, "Content-Disposition: form-data; name=\"audio\"; "
            + "filename=\"recording.wav\"\r\n");
write(body, "Content-Type: audio/wav\r\n\r\n");
body.write(audio);
write(body, "\r\n--" + boundary + "--\r\n");
```

The field name `audio` must match the parameter name on the server. The request is sent with:

```
HttpClient client = HttpClient.newBuilder()
        .version(HttpClient.Version.HTTP_1_1)
        .connectTimeout(Duration.ofSeconds(10))
        .build();
```

The request itself has a 180-second timeout, because the first request after starting the server can take a long time. Forcing HTTP/1.1 solved a real integration problem described in Chapter 7.

6.3.6 Background thread with SwingWorker

Network calls must never run on Swing's event thread, or the window would freeze. The method `sendToBackend()` creates a `SwingWorker<String, Void>`. Its `doInBackground()` builds and sends the request and returns the JSON text. Its `done()` method runs on the event thread, so it can safely update the text areas. In `done()`, a `ConnectException` is detected and turned into the friendly message that tells the user to start the Python server. The `finally` block deletes the temporary WAV file.

6.3.7 Reading the JSON reply

The reply has a fixed, simple shape, so a small method `jsonValue(json, key)` was written instead of adding a JSON library. It finds the key, skips the colon and spaces, then reads characters up to the closing quote while decoding the escape sequences `\n`, `\r`, `\t`, `\b`, `\f` and `\uXXXX`. The `\uXXXX` case is important, because JSON servers may send Devanagari characters as Unicode escapes.

6.4 Implementation of the Python Server

6.4.1 Application and start-up

The FastAPI application is created with a `lifespan` function. Code before the `yield` runs once when the server starts; code after it runs at shutdown. Loading the model there means it is loaded one time and then reused for every request, which is essential because loading takes much longer than recognizing a short clip.

6.4.2 Preparing the model folder

During development a problem appeared with the Hugging Face cache (Section 7.5). The final code therefore keeps the model in a normal folder, `model_files`, next to `main.py`:

1. If `model_files` already contains `config.json` and `model_onnx.py`, it is used directly, with no internet and no access check. This also allows the folder to be copied from one computer to another.
2. Otherwise `huggingface_hub.snapshot_download(MODEL_ID, local_dir=local_dir)` downloads the repository into it.
3. Any symbolic link found in the folder is replaced by a real copy of the file.
4. The model's own loader calls `snapshot_download` internally, so that function is replaced by a small wrapper that returns the local folder when it is asked for this model.
5. `AutoModel.from_pretrained(..., trust_remote_code=True)` then loads the model. The option `trust_remote_code` is needed because the repository contains its own Python class (`model_onnx.py`) that defines how the exported model is run.

6.4.3 Reading and converting audio

```
data, sr = sf.read(io.BytesIO(raw), dtype="float32", always_2d=True)
mono = data.mean(axis=1)
wav = torch.from_numpy(np.ascontiguousarray(mono)).unsqueeze(0)
if sr != TARGET_SR:
    wav = AF.resample(wav, sr, TARGET_SR)
```

`always_2d=True` gives a (samples, channels) array whatever the file, so averaging over axis 1 always produces a mono signal. The tensor shape (1, samples) is the batch format the model expects. Because the client records at 44,100 Hz, the resampling step runs for every request. If the file cannot be read, the function raises an HTTP 400 error.

6.4.4 Running recognition

```
with torch.inference_mode():
    result = state["model"](wav, LANG_CODE, DECODER)
```

The model object is called with the waveform, the language code `kok` and the decoder name. `torch.inference_mode()` disables gradient bookkeeping, which saves memory and time. The result is a string (or a list whose first item is a string) and is stripped of surrounding spaces.

6.4.5 Building the reply

The endpoint returns a Python dictionary with the keys `devanagari` and `roman`, and FastAPI converts it to JSON. Before running the model, the endpoint checks that the model is loaded (otherwise HTTP 503) and that the upload is not tiny (otherwise HTTP 400).

6.5 Implementation of Transliteration

The function `_transliterate_word()` walks through the characters of one word with an index `i`. The central branch handles consonants:

```
if ch in CONSONANTS:
    base = CONSONANTS[ch]
    i += 1
    if i < n and word[i] == NUKTA: i += 1
    if i < n and word[i] == VIRAMA:            # half consonant
        out.append(base); i += 1
    elif i < n and word[i] in MATRAS:          # explicit vowel sign
        out.append(base + MATRAS[word[i]]); i += 1
    else:                                      # inherent vowel
        at_word_end = i >= n or not _is_letter(word[i]) or word[i] in SIGNS
        if at_word_end and not (i < n and word[i] in SIGNS):
            out.append(base)
        else:
            out.append(base + "a")
```

Other branches handle independent vowels, anusvara and visarga, digits, and pass-through of other characters. `to_roman()` splits the sentence on spaces and applies the word function to each part. The tables are plain Python dictionaries, so adding or changing a mapping, for example writing ी as `i` instead of `ee`, takes a single edit.

6.6 Configuration

Table 6.2: *Configuration settings*

Setting	How to change	Default
Model name	Environment variable `KONKANI_MODEL`	`ai4bharat/indic-conformer-600m-multilingual`
Decoder	Environment variable `KONKANI_DECODER`	`ctc` (use `rnnt` for a slower, often more accurate decoder)
Model folder	Environment variable `KONKANI_MODEL_DIR`	`model_files` next to `main.py`
Server address and port	Command line of Uvicorn	`127.0.0.1`, `8000`
Client server address	Constant in the Java source	`http://127.0.0.1:8000/transcribe`

6.7 How to Run

1. Install the Python packages once (`pip install -r requirements.txt`) with Python 3.12.
2. Start the server: `uvicorn main:app --host 127.0.0.1 --port 8000` and wait for the message “Model loaded.”
3. In a second terminal compile and start the client: `javac -encoding UTF-8 KonkaniLiveSpeechApp.java` then `java KonkaniLiveSpeechApp`.
4. Click **Start recording**, speak, click **Stop & transcribe**.

CHAPTER 7

Problems Faced During Development and Their Solutions

A large part of the effort in this project went into making the software run reliably on different computers. This chapter records each significant problem, how it was diagnosed and how it was solved. We include it because it shows the practical engineering involved in using a modern deep-learning model, and because the same problems will be met by anyone repeating the project.

7.1 Summary Table

Table 7.1: *Summary of problems, causes and solutions*

No.	Problem	Cause	Solution
P1	Requirement described as “text to speech”	Wording reversed	Redefined as speech-to-text
P2	`ModuleNotFoundError` for fastapi, numpy, soundfile	Packages not installed for the Python that ran the file	Install `requirements.txt` with the right interpreter; start with Uvicorn
P3	ML libraries unsuitable on the newest Python	Python 3.13 and later not yet supported by the packages	Use Python 3.12
P4	Virtual environment could not be created on Windows	Copy error for `venvlauncher.exe` (permissions or security software)	Use a short path outside Downloads, or install without a virtual environment
P5	Segmentation fault when installing pip on macOS	Damaged Python installation	Reinstall Python and recreate the environment
P6	Model download refused with HTTP 401	Model is gated; not logged in	Hugging Face account, token, login
P7	Model download refused with HTTP 403	Access request not submitted or wrong account	Submit the request; use a token from the account that has access
P8	ONNX Runtime error “External data path escapes model directory”	Cache files are symbolic links	Use a normal model folder with real files
P9	Model loader rejected a local path	Loader treats its argument as a repository name	Wrap `snapshot_download` to return the local folder
P10	HTTP 422, field `audio` missing	Java’s HTTP/2 upgrade request confused Uvicorn	Force HTTP/1.1 in the Java client
P11	Java virtual machine crash, “insufficient memory”	Windows had almost no free RAM or paging space	Restart, close heavy programs, enlarge paging file

No.	Problem	Cause	Solution
P12	Old behaviour after fixing the code	Unsaved file and a stale build from the editor's Run button	Save, delete `.class` files, compile and run from the terminal
P13	Alternative backend with NeMo and IndicXlit did not install	Heavy dependencies not supported on Windows	Use the Transformers-based model and rule-based transliteration

7.2 P2 and P3: Missing Modules and the Python Version

The first error reported was `No module named 'fastapi'`, later numpy and soundfile. The cause was always the same: the file had been started with a Python interpreter that did not have the packages. Several Pythons were present on the same computer (3.10, 3.12, 3.13 and an Anaconda installation), and the editor's Run button chose one of them. Two lessons followed. First, install the packages with the same interpreter that will run the program, for example `py -3.12 -m pip install -r requirements.txt`. Second, a FastAPI program must be started through Uvicorn (`uvicorn main:app`), not by running `main.py` as a script, because running the file only defines the application and never starts the web server.

Python 3.13 was installed by default, but the machine-learning packages are normally released for older versions first. Using Python 3.12 removed installation failures.

7.3 P4 and P5: Environment Creation Failures

On Windows, `python -m venv venv` failed with a message that `venvlauncher.exe` could not be copied. Likely causes are an antivirus program, a half-created folder from an earlier attempt, or a restricted location such as the Downloads folder. The fixes were to delete the broken `venv` folder, work from a short path such as `C:\konkani`, use Python 3.12 explicitly, and, if necessary, skip the virtual environment and install into Python 3.12 directly.

On the Mac, pip crashed with a segmentation fault while the environment was being created. This points to a damaged Python build. Installing a clean Python and recreating the environment with `rm -rf venv` and `python3.12 -m venv venv` solved it.

7.4 P6 and P7: Gated Model Access

The IndicConformer repository is gated: Hugging Face allows downloads only for accounts that have accepted the model's terms. The first attempt returned HTTP 401 Unauthorized, meaning the server did not know who was asking. After logging in with an access token the error changed to HTTP 403 Forbidden, which meant the account was known but not on the approved list. The page `settings/gated-repos` showed that no access request had ever

been made. Submitting the form on the model page granted access, and the download then succeeded.

When the project was later moved to a second computer, the same 403 appeared again, because the token on that computer belonged to a different Hugging Face account. Access belongs to the account, not to the computer. Using a token created from the account that had access, or copying the downloaded `model_files` folder, solved the problem. The offline mode described in Section 6.4.2 was added for this reason.

7.5 P8 and P9: ONNX Runtime Path Check

After the download, loading failed with `External data path escapes model directory`. Hugging Face stores each downloaded file in a hidden `blobs` folder and places a symbolic link in the model folder. Newer ONNX Runtime versions check that a weight file lives inside the model directory, find that the link points outside it, and refuse to load it for security reasons. The first attempt to fix this passed a local folder to `from_pretrained`, but the model's own loader code calls `snapshot_download(repo_id=...)` on whatever it receives, and rejected a file path as an invalid repository name. The final solution has three parts: download into a normal folder, convert links to real files, and intercept the loader's call so that it returns this folder (Section 6.4.2). A second possible fix is to install an older ONNX Runtime version.

7.6 P10: HTTP 422 and the Java Upload

When the client first reached the server it received HTTP 422 with the message `Field required` for `body.audio`, and the Uvicorn log showed `Unsupported upgrade request` and `Invalid HTTP request received`. A 422 means the request was understood but a required field was missing, so the file was not arriving. Java's `HttpClient` tries by default to upgrade a plain HTTP connection to HTTP/2, and Uvicorn does not support that upgrade for requests with a body, so the body was lost. Adding `.version(HttpClient.Version.HTTP_1_1)` made the client use ordinary HTTP/1.1 and the upload arrived correctly. A related client message, `fixed content-length: 90, bytes received: 0`, was another symptom of the same dropped connection.

7.7 P11: Out-of-Memory Crash of the Java Virtual Machine

One run produced a long crash report from the Java virtual machine, ending with `There is insufficient memory for the Java Runtime Environment to continue`. The report showed the computer had about 14 GB of RAM but only about 0.6 GB free, with almost no paging space left, after more than 70 days without a restart and with several large programs open. The server's model needs several gigabytes, so the system must have enough free memory. The remedies were to restart the computer, close browsers and other

development tools before starting, start the small Java client first if memory is short, and enlarge the Windows paging file.

7.8 P12: Stale Builds and Unsaved Files

After the HTTP/1.1 fix had been written, the error continued. The editor tab showed an unsaved-changes dot, and the editor's Run button was using its own earlier compiled copy. Saving the file, deleting old `.class` files and running `javac` and `java` from the terminal ensured the new code ran.

7.9 Lessons Learned

1. Check which interpreter and which build are really running before debugging the code.
2. Read error codes carefully: 401, 403 and 422 each point to a different layer.
3. Keep the interface between programs small and test it with a simple client first.
4. Large models make memory and disk space part of the design.
5. Document every environment step, because installation, not coding, is often the hardest part of an applied machine-learning project.

CHAPTER 8

Testing and Evaluation

8.1 Test Strategy

Testing was planned at four levels.

Table 8.1: *Levels of testing*

Level	What is tested	Method
Unit	The transliteration function and the audio conversion function	Run on sample inputs and compare with expected outputs
Component	The server endpoints on their own	Open `/health` in a browser; send a WAV file with a test client
Integration	Java client and Python server together	Record a clip in the client and check the displayed text
System / acceptance	The complete product in realistic use	Native speakers read prepared sentences; results are checked against the written text

8.2 Test Environment

Table 8.2: *Test environment*

Item	Setting
Computer 1	Windows 11, AMD Ryzen 7, about 14 GB RAM, no discrete GPU
Computer 2	MacBook Air with Apple silicon
Python / Java	Python 3.12; JDK 17 or later
Microphone	Built-in laptop microphone and a wired headset
Room	Quiet room, speaker about 20 to 30 cm from the microphone
Server mode	CPU only, CTC decoder

8.3 Functional Test Cases

The table lists the test cases, the input, the expected result and a column in which the tester records the outcome after running the test. Results that depend on the tester’s own recordings are left to be filled in at execution time.

Table 8.3: *Functional test cases*

ID	Test	Input / action	Expected result	Outcome (Pass / Fail, date)
TC1	Server start	Run `uvicorn main:app`	Message “Model loaded.” and Uvicorn running on port 8000	
TC2	Health check	Open `http://127.0.0.1:8000/health`	`{"status":"ok","model_loaded":true}`	
TC3	Client start	Run `java KonkaniLiveSpeechApp`	Window opens; Start enabled, Stop disabled	
TC4	Start recording	Click Start recording	Status shows “Recording…”; Start disabled; Stop enabled	
TC5	Normal transcription	Speak a Konkani sentence for 5–10 s, click Stop	Both boxes show text; status “Done”	
TC6	Very short recording	Click Start, then Stop immediately	Status “Recording too short. Try again.”; no request sent	
TC7	Silence	Record 5 s of silence	No crash; empty or very short text	
TC8	Server not running	Close the server, then record and click Stop	Dialog “Cannot reach the Python backend”	
TC9	Server still loading	Record while the server is starting	Dialog showing an error from the server or a connection message	
TC10	Microphone denied	Disable microphone access, click Start	Dialog “Could not start microphone recording”	
TC11	Repeated use	Record and transcribe three times in a row	Each run works; previous text is cleared at each start	
TC12	Long recording	Record 30 s	Text returned; no crash; time noted	
TC13	Temporary file removal	Check the temp folder after a run	The file `konkani-mic-*.wav` is deleted	
TC14	Unreadable upload	Send a text file as audio to `/transcribe`	HTTP 400 with a message	
TC15	Missing field	Send a request without the `audio` part	HTTP 422	
TC16	Offline start	Disconnect the internet and start the server with `model_files` present	Server starts and loads from the local folder	

8.4 Unit Tests of the Transliteration Module

The transliteration function was executed on sample words, and the outputs below are the real results.

Table 8.4: *Unit tests of the transliteration module*

Input	Expected	Actual output	Result
कमल	kamal	kamal	Pass
नमस्कार	namaskaar	namaskaar	Pass
देव	dev	dev	Pass
घर	ghar	ghar	Pass
मोगो	mogo	mogo	Pass
बरें	baren	baren	Pass
कितें	kiten	kiten	Pass
म्हजें	mhajen	mhajen	Pass
गोंय	gony	gony	Pass
शाळा	shaalaa	shaalaa	Pass
देव बरें तुमी (three words)	dev baren tumee	dev baren tumee	Pass

> **Note.** The “expected” column describes what the rule table is designed to produce. A Konkani writer may prefer a different Roman spelling for some words (for example *tumi* for तुमी); this is a property of the chosen rules, not a program error.

8.5 Accuracy Evaluation Method

The quality of recognition is measured with the Word Error Rate (Section 2.5). The following method is used.

1. **Prepare test sentences.** Select a set of short Konkani sentences (for example 30) in Devanagari that cover everyday vocabulary, numbers and names.
2. **Record.** Ask speakers of different dialects, such as Goan and Mangalorean, to read the sentences, at least three speakers per dialect, with their consent.
3. **Transcribe with the system.** Run each recording through the application and save the Devanagari output.
4. **Compare.** Align each output with the reference sentence and count substitutions (S), deletions (D) and insertions (I).
5. **Compute.** WER = (S + D + I) / N for each sentence, then the average per dialect. Do the same at character level for CER.

6. **Report.** Present the results in a table such as the one below and discuss the differences between dialects.

> **Worked example (illustration only).** Suppose the reference has six words and the system output has one wrong word and one missing word, with no extra words. Then S = 1, D = 1, I = 0, N = 6 and WER = (1 + 1 + 0) / 6 = 0.33, or 33 %.

Result table to be completed after the recordings:

Table 8.5: *Word and character error rates by dialect (to be completed)*

Dialect	Speakers	Sentences	Words (N)	WER	CER
Goan Konkani					
Mangalorean Konkani					
All					

Because public data for Mangalorean Konkani is smaller, the expected finding is a higher error rate for that dialect. The measured values will show the size of the gap.

8.6 Performance Measurement

Table 8.6: *Performance measurements (to be recorded)*

Measurement	How to record it	Value
Model load time at start-up	Time from launching Uvicorn to “Model loaded.”	
First transcription time	Time from clicking Stop to text appearing, first request	
Later transcription time (5 s clip)	Same, after warm-up	
Later transcription time (30 s clip)	Same	
Server memory use	Task Manager or Activity Monitor while idle and while transcribing	

8.7 Test Observations from Development

The following was observed while the system was being built and is documented in Chapter 7: the server started and reported that the model was loaded on both computers once the environment issues were fixed; the client produced the intended error dialogs when the upload failed; and the client’s behaviour changed after forcing HTTP/1.1. These observations are the basis for the error-handling tests TC8 to TC10 and TC14 to TC15.

Two real transcription results from our test runs (from the Java window on the MacBook): one recording produced the Devanagari text हेलो हेलो हेलो with the Roman text “helo helo helo”, and another produced कितें करते with the Roman text “kiten karate”. These show the whole chain working from microphone to both outputs. They are short informal tests, not a measured accuracy figure.

CHAPTER 9

Working Product and Screenshots

9.1 Working Product Screenshot

The figure below shows the Java client running on the MacBook Air after two test recordings. The top box shows the recognized Devanagari text and the bottom box shows the machine-transliterated Roman text. The window on the left shows हेलो हेलो हेलो with “helo helo helo”, and the window on the right shows कितें करते with “kiten karate”. The status line reads “Done. Please check spelling and transliteration.”

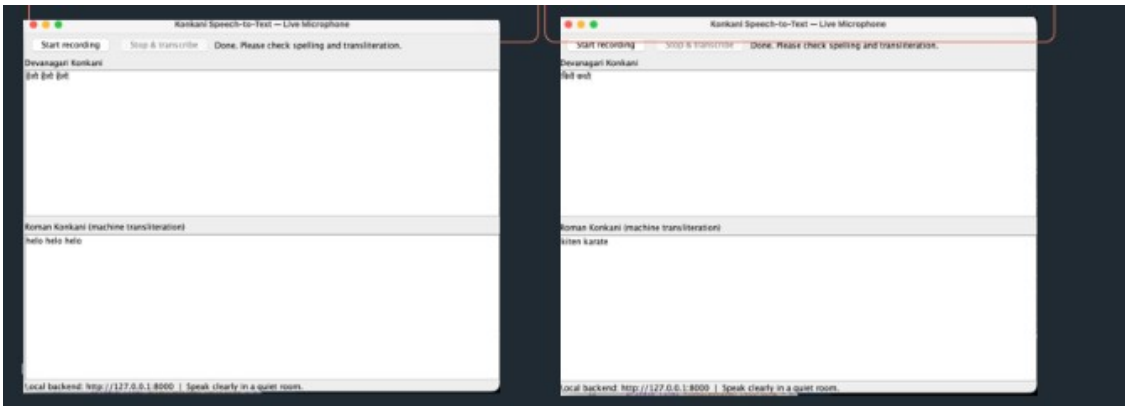


Figure 9.1: *The Konkani Speech-to-Text client showing two test recordings*

CHAPTER 10

Limitations, Future Scope and Conclusion

10.1 Limitations

1. **Dialect accuracy.** The model's training data contains more Goan than Mangalorean Konkani, so recognition of Mangalorean speech and of local vocabulary is expected to be weaker. A published evaluation of this same model on a Konkani community speech set (Konkani Daan, WILDRE 2026) reported a word error rate of about 46.5 %, so errors should be expected even for clear speech.
2. **No Roman spelling standard.** The Roman output is a fixed rule-based transliteration. It does not know the spelling habits of individual writers and writes long vowels as *aa* and *ee*.
3. **Medial schwa not removed.** The rules drop the final inherent "a" of a word but not the one inside a word, so some words are written with an extra *a*.
4. **Not real time.** Recognition starts after the user stops recording. There is no word-by-word streaming.
5. **Hardware demands.** The model needs several gigabytes of memory and disk space, and the first request is slow.
6. **Setup effort.** The user must install Python packages, accept the model's terms and download a large file once.
7. **Noise sensitivity.** Background noise and a poor microphone reduce accuracy.
8. **Single user, single computer.** The server listens on the loopback address only.
9. **Script coverage.** Kannada and Malayalam scripts for Konkani are not produced.

10.2 Future Scope

Table 10.1: *Future enhancements and their benefits*

Enhancement	Benefit
Fine-tune the model on recorded Mangalorean Konkani speech	Better accuracy for the under-represented dialect
Build a Konkani speech corpus with consent from speakers	Data for training and for a fair evaluation
Streaming recognition with an RNN-T model	Text appears while the user speaks
Better transliteration (schwa deletion in the middle of words, learned spelling, user-selectable styles)	More natural Roman Konkani
Android and web front ends using the same API	Wider reach
Kannada-script output	Serves the Mangalore region
Add a text-correction step and a Konkani language model	Fewer spelling errors

Enhancement	Benefit
Package the server as a single installer	Easier setup for non-technical users
Voice activity detection	Automatic start and stop of recording

10.3 Learning Outcomes

During this project we learned:

- how a speech-recognition pipeline works from the microphone to the text, including sampling, resampling and neural decoding;
- how to design and build a client–server application with a clean API between a Java GUI and a Python service;
- how to use threads and `SwingWorker` to keep a desktop user interface responsive;
- how to build a multipart HTTP request by hand and how to read HTTP status codes;
- how to use a gated model from Hugging Face, including accounts, tokens and offline use;
- how to diagnose environment problems in Python, Java and the operating system; and
- how to evaluate a speech system with Word Error Rate and why dialect coverage matters for low-resource languages.

10.4 Conclusion

The project achieved its aim of building a working application in which a person speaks Konkani into a microphone and sees the words as Devanagari and Roman text. By choosing a pretrained AI4Bharat model, a thin Java client and a small rule-based transliterator, a task that would otherwise need months of data collection and training became feasible as a mini project. The brainstorming and the comparison of options led to a design that is simple, runs offline after the first download, and keeps the user interface separate from the machine-learning code.

The work also showed where the difficulties lie. Installation and compatibility problems cost as much effort as the program itself, and the quality for the Mangalorean dialect depends on data that is not yet widely available. These limits point directly to the next steps: collect and share more Konkani speech, fine-tune the model, and improve the transliteration. We hope this project is a useful first step towards everyday speech technology for Konkani.

REFERENCES

[1] A. Graves, S. Fernández, F. Gomez and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proc. International Conference on Machine Learning (ICML)*, 2006.

[2] A. Graves, "Sequence transduction with recurrent neural networks," arXiv:1211.3711, 2012.

[3] A. Gulati et al., "Conformer: Convolution-augmented Transformer for Speech Recognition," in *Proc. Interspeech*, 2020.

[4] A. Baevski, H. Zhou, A. Mohamed and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] A. Radford et al., "Robust speech recognition via large-scale weak supervision," OpenAI, 2022.

[6] T. Javed et al., "Towards building ASR systems for the next billion users," in *Proc. AAAI Conference on Artificial Intelligence*, 2022.

[7] T. Javed et al., "IndicVoices: Towards building an inclusive multilingual speech dataset for Indian languages," *Findings of the Association for Computational Linguistics (ACL)*, 2024.

[8] AI4Bharat, "IndicConformer 600M multilingual speech recognition model," Hugging Face: `ai4bharat/indic-conformer-600m-multilingual`.

[9] Konkani Daan Konkani community speech set and its evaluation, WILDRE workshop, 2026.

[10] Software documentation: FastAPI (fastapi.tiangolo.com), PyTorch and TorchAudio (pytorch.org), Hugging Face Transformers (huggingface.co/docs/transformers), ONNX Runtime (onnxruntime.ai), SoundFile (python-soundfile.readthedocs.io).

APPENDIX A

Java Client Source Code

File: `java/KonkaniLiveSpeechApp.java`

> **To be added.** Paste the complete source code of KonkaniLiveSpeechApp.java here (use the Courier New font, size 8–9).

APPENDIX B

Python Server Source Code

File: `py/main.py`

> **To be added.** Paste the complete source code of main.py here (use the Courier New font, size 8–9).

APPENDIX C

API Specification

The Python server exposes two endpoints on `http://127.0.0.1:8000`.

Table 10.2: *API endpoints*

Method	Path	Description
GET	`/health`	Health check. Returns `{"status":"ok","model_loaded":true}` when the model has been loaded.
POST	`/transcribe`	Accepts a WAV file in the multipart/form-data field `audio`, converts it to 16 kHz mono, recognizes Konkani speech and returns the Devanagari and Roman text as JSON.

Response of `POST /transcribe` (HTTP 200):

```
{
  "devanagari": "<recognized text in Devanagari>",
  "roman": "<machine transliteration in Roman script>"
}
```

Table 10.3: *Status codes returned by POST /transcribe*

Code	Meaning	Typical cause
200	Success	Both texts returned
400	Bad request	Audio too short or the file cannot be read as audio
422	Validation error	The `audio` field is missing from the request
503	Service unavailable	The model is not loaded yet